# *Robot Navigation Using Q-Learning: Effect of Reward Design on the Learned Path*

Jenn Price
*MTRE 4820 Machine Learning for Robot Perception*
*Kennesaw State University*
Marrietta, United States
10/29/2026

***Abstract*—This report applies tabular Q-learning to the in-class robot navigation problem, in which a robot must learn to leave a building of five connected rooms (0–4) and reach the outside area, room 5. The environment is encoded as a reward matrix R, and a MATLAB program learns the action-value matrix Q using a discount factor γ = 0.8 and a learning rate α = 1. With a reward of 100 for every move into room 5, the learned policy finds two equally good three-move routes from room 2 (2→3→1→5 and 2→3→4→5). When the reward for the move from room 1 to room 5 is raised to 200, the tie is broken and the learned path becomes 2→3→1→5; the larger reward also makes rooms 0 and 4 abandon their direct exit in favor of a longer route through room 1. These results show that the reward function, not the path length alone, determines the behavior that the agent learns.**

***Keywords—reinforcement learning, Q-learning, robot navigation, reward design, MATLAB***

## I. Introduction

Reinforcement learning (RL) lets an agent learn a behavior by trial and error: it acts in an environment, receives a numerical reward, and gradually prefers the actions that lead to the largest long-term reward [1], [2]. Q-learning is a model-free RL algorithm that learns a table Q(s, a) giving the value of taking action *a* in state *s* and then acting optimally afterwards [1]. Once the table has converged, the robot only needs to choose the action with the largest Q value in each state.

In this project, Q-learning is used to teach a robot to navigate out of the building shown in the project handout. The objectives are to (1) define the environment and rewards, (2) implement Q-learning in MATLAB, (3) find the optimal path from room 2 to room 5, and (4) study how changing the reward from room 1 to room 5 from 100 to 200 changes what the robot learns.

## II. Environment and Reward Matrix

Each room is a state, s ∈ {0, 1, 2, 3, 4, 5}, where room 5 is the outside area (Finish) and is the goal state. An action is the choice of which door to go through, so an action is identified by the room it leads to. From the floor plan, the doors are 0–4, 1–3, 1–5, 2–3, 3–4, and 4–5. All doors can be used in both directions, and a self-loop 5–5 is added so that the goal is absorbing. Room 2 has only one door, so the robot starting in room 2 must first move to room 3.

The rewards are stored in the 6×6 matrix R, where the row is the current room and the column is the next room. An entry of −1 means there is no door (an illegal move), 0 means a legal move that does not reach the goal, and 100 means a legal move into room 5. Table I shows the baseline reward matrix used in Part 1. In Part 2 the single entry R(1, 5) is changed from 100 to 200 and all other entries are unchanged.

TABLE I. Reward Matrix R for Part 1

| s \ a | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| **0** | −1 | −1 | −1 | −1 | 0 | −1 |
| **1** | −1 | −1 | −1 | 0 | −1 | 100 |
| **2** | −1 | −1 | −1 | 0 | −1 | −1 |
| **3** | −1 | 0 | 0 | −1 | 0 | −1 |
| **4** | 0 | −1 | −1 | 0 | −1 | 100 |
| **5** | −1 | 0 | −1 | −1 | 0 | 100 |

## III. Q-Learning Implementation

The Q matrix is initialized to zero, and the agent learns from episodes. In every episode a random non-goal starting room is chosen, and the robot keeps moving until it reaches room 5. At each step it selects one of the legal actions of the current room at random, observes the next state s′, and updates the Q value with

$$Q_{k+1}(s, a) = (1 - \alpha)\, Q_k(s, a) + \alpha\, [R(s, a) + \gamma \max_{a'} Q_k(s', a')]$$

With α = 1, as required, this reduces to the update in the project handout, Q(s, a) = R(s, a) + γ·max Q(s′, a′). The next state then becomes the current state. Because room 5 ends the episode, its row of Q is never updated and stays zero. The discount factor γ = 0.8 means that a reward received k moves in the future is worth $0.8^k$ of its face value.

The MATLAB program (Project7_QLearning.m) is written as one main function with local functions: buildR creates R from a list of doors, qLearn performs the training, allOptimalPaths extracts the greedy path, and two plotting functions draw the convergence curve and the path. Three improvements were made over a basic implementation: (i) the random number generator is seeded (rng(4820)) so the results are reproducible, (ii) training stops automatically once Q has not changed for 100 consecutive episodes instead of using a fixed, arbitrary number of episodes, and (iii) the path search follows every action that ties for the maximum Q value, so that equally good routes are reported instead of hidden.

## IV. Results

### A. Learned Q Matrix in the First Iterations

Tables II–IV show Q after the first three episodes. The first episode wandered 3→2→3→2→3→4→5. Since every Q value was still zero, the only update that produced a non-zero value was the last one, Q(4, 5) = 100 + 0.8·0 = 100. In episode 2 (2→3→4→0→4→5) the value of room 4 was passed back one step, giving Q(3, 4) = Q(0, 4) = 0.8·100 = 80. In episode 3 the values spread further back to Q(4, 0) = Q(1, 3) = 64 and Q(3, 1) = 51.2. This shows how Q-learning propagates the goal reward backward through the building one step at a time. With the same random seed, the first three episodes of Part 2 follow the same moves and never take the 1→5 door, so their Q matrices are identical to Tables II–IV.

TABLE II. Q Matrix after Episode 1

| s \ a | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 0 | 0 |
| **1** | 0 | 0 | 0 | 0 | 0 | 0 |
| **2** | 0 | 0 | 0 | 0 | 0 | 0 |
| **3** | 0 | 0 | 0 | 0 | 0 | 0 |
| **4** | 0 | 0 | 0 | 0 | 0 | 100 |
| **5** | 0 | 0 | 0 | 0 | 0 | 0 |

TABLE III. Q Matrix after Episode 2

| s \ a | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 80 | 0 |
| **1** | 0 | 0 | 0 | 0 | 0 | 0 |
| **2** | 0 | 0 | 0 | 0 | 0 | 0 |
| **3** | 0 | 0 | 0 | 0 | 80 | 0 |
| **4** | 0 | 0 | 0 | 0 | 0 | 100 |
| **5** | 0 | 0 | 0 | 0 | 0 | 0 |

TABLE IV. Q Matrix after Episode 3

| s \ a | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 80 | 0 |
| **1** | 0 | 0 | 0 | 64 | 0 | 0 |
| **2** | 0 | 0 | 0 | 0 | 0 | 0 |
| **3** | 0 | 51.2 | 0 | 0 | 80 | 0 |
| **4** | 64 | 0 | 0 | 0 | 0 | 100 |
| **5** | 0 | 0 | 0 | 0 | 0 | 0 |

Fig. 1 shows the sum of all Q values per episode. The Q matrix stopped changing after episode 5 in Part 1 and after episode 7 in Part 2; the program then confirmed convergence over 100 further episodes and stopped at episodes 105 and 107, respectively.

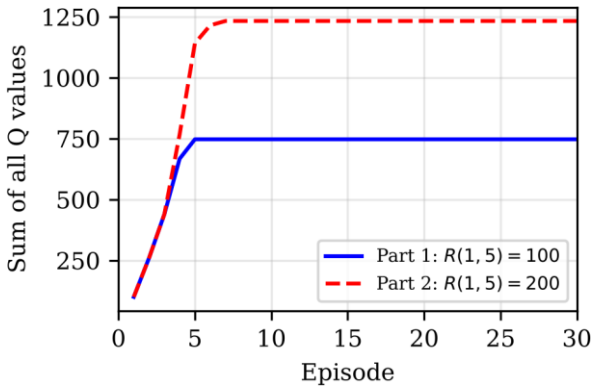



Fig. 1. Convergence of the Q matrix (first 30 episodes shown) for Part 1 and Part 2.

### B. Part 1: Final Q Matrix and Optimal Path

Table V shows the final Q matrix for the baseline rewards. The values follow the pattern $100 \cdot 0.8^n$, where n is the number of moves after the first one needed to reach the goal: moves directly into room 5 are worth 100, moves one step away are worth 80, and so on. The optimal path is found by starting in room 2 and repeatedly choosing the action with the largest Q value. From room 2 the only option is room 3 (Q = 64). In room 3, Q(3, 1) = Q(3, 4) = 80, so two optimal paths exist, each with three moves:

$$2 \rightarrow 3 \rightarrow 1 \rightarrow 5 \quad \text{and} \quad 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$$

Both routes give exactly the same discounted reward, so the robot may take either one (MATLAB's max function would pick room 1, the first index).

TABLE V. Final Q Matrix, Part 1 (R(1,5) = 100)

| s \ a | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 80 | 0 |
| **1** | 0 | 0 | 0 | 64 | 0 | **100** |
| **2** | 0 | 0 | 0 | **64** | 0 | 0 |
| **3** | 0 | **80** | 51.2 | 0 | **80** | 0 |
| **4** | 64 | 0 | 0 | 64 | 0 | **100** |
| **5** | 0 | 0 | 0 | 0 | 0 | 0 |

### C. Part 2: Final Q Matrix and Optimal Path

Table VI shows the final Q matrix after R(1, 5) is changed to 200. Every value on a route that passes through the 1→5 door has doubled: Q(1, 5) = 200, Q(3, 1) = 160, and Q(2, 3) = 128. Room 3 now clearly prefers room 1 (160) over room 4 (102.4), so the tie is broken and the optimal path from room 2 is unique:

$$2 \rightarrow 3 \rightarrow 1 \rightarrow 5$$

TABLE VI. Final Q Matrix, Part 2 (R(1,5) = 200)

| s \ a | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 102.4 | 0 |
| **1** | 0 | 0 | 0 | 128 | 0 | **200** |
| **2** | 0 | 0 | 0 | **128** | 0 | 0 |
| **3** | 0 | **160** | 102.4 | 0 | 102.4 | 0 |
| **4** | 81.92 | 0 | 0 | 128 | 0 | 100 |
| **5** | 0 | 0 | 0 | 0 | 0 | 0 |

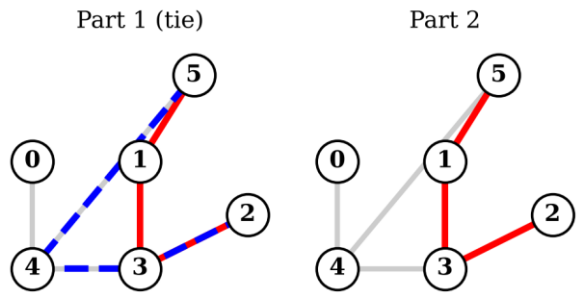



Fig. 2. Optimal path(s) from room 2 to room 5. Part 1 has two equally good paths (solid and dashed); Part 2 has a single path through room 1.

## V. Effect of the Reward on Learning Results

The comparison of Parts 1 and 2 shows that the reward matrix directly shapes the policy the robot learns. Since Q values are discounted sums of rewards, a change to one reward is passed backward to every state that can reach it, scaled by $\gamma$ for each extra move. Doubling R(1, 5) doubled every Q value that depends on that door, while values that only depend on the 4→5 exit, such as Q(4, 5) = 100, did not change.

In Part 1 both exits were worth the same, so the policy only cared about the number of moves, and two equally short paths tied. In Part 2 the larger reward broke the tie in favor of room 1. The more interesting effect appears in rooms 4 and 0. In Part 1, the best action from room 4 is the direct exit to room 5 (Q = 100). In Part 2, however, $Q(4, 3) = 200 \cdot 0.8^2 = 128$ is larger than Q(4, 5) = 100, so the robot in room 4 now prefers the three-move detour 4→3→1→5 over the one-move exit, and room 0 follows the same route (0→4→3→1→5). This happens because the detour is preferred whenever $R(1, 5) \cdot \gamma^2 > 100$, i.e., R(1, 5) > 156.25.

This shows that Q-learning maximizes the total discounted reward, not the number of moves. A designer who only wanted to make room 1's exit slightly more attractive could accidentally make the robot take a much longer route. The discount factor controls this trade-off: a smaller $\gamma$ penalizes long paths more strongly, while a $\gamma$ close to 1 makes the robot chase the largest reward almost regardless of distance. The reward function and $\gamma$ must therefore be chosen to match the behavior that is actually desired.

## VI. Conclusion

A MATLAB Q-learning program was developed for the in-class robot navigation problem with $\gamma = 0.8$ and $\alpha = 1$. The Q matrix converged within a few episodes and was confirmed over 100 additional episodes. With a reward of 100 for entering room 5, the robot learned two equally optimal routes from room 2, 2→3→1→5 and 2→3→4→5. Increasing the reward from room 1 to room 5 to 200 made 2→3→1→5 the single optimal route and also caused rooms 0 and 4 to prefer a longer path through room 1. The experiment shows that the learned behavior depends strongly on how the rewards are designed.